

2. Requisitos del software

Este capítulo presenta un análisis exhaustivo de los requisitos del sistema para esta aplicación que, como ya hemos descrito, trataremos de desarrollar una plataforma educativa de evaluación automática de ejercicios de programación. Partiendo del contexto de la introducción y siguiendo las mejores prácticas en ingeniería de requisitos, se detallan las especificaciones funcionales, no funcionales y de información necesarias para el desarrollo del sistema, así como los casos de usos que los distintos usuarios de la plataforma puedan encontrarse.

Este análisis constituye la base fundamental para las etapas posteriores de diseño e implementación, asegurando que el sistema desarrollado responda adecuadamente a las necesidades identificadas como tal.

2.1. Introducción a la ingeniería de requisitos

La especificación de requisitos de software constituye una descripción completa del comportamiento que debe exhibir el sistema a desarrollar. En el contexto de esta plataforma educativa, los requisitos deben reflejar tanto las necesidades pedagógicas de los profesores y los estudiantes, como también los desafíos técnicos adheridos a la ejecución segura de código arbitrario en un entorno compartido.

La ingeniería de requisitos para este proyecto se fundamenta en los principios establecidos por el estándar IEEE 830-1998, que define las características esenciales de una especificación de requisitos de calidad: debe ser completa, consistente inequívoca, correcta, trazable, priorizable, modificable y verificable. Estos atributos garantizan que todos los aspectos relevantes del sistema queden adecuadamente especificados y que exista una comprensión común entre todas las partes interesadas del mismo. [\[7\]](#) [\[8\]](#)

Teniendo en cuenta lo anterior, los requisitos se organizan en tres categorías principales que cubren diferentes aspectos del sistema que vamos a desarrollar:

- **Requisitos funcionales:** Especifican las acciones, comportamientos y funciones concretas que debe realizar el sistema, describiendo qué debe hacer la plataforma en respuesta a entradas específicas.
- **Requisitos no funcionales:** Definen atributos de calidad que determinan cómo debe comportarse el sistema, incluyendo aspectos como rendimiento, seguridad, usabilidad y escalabilidad.
- **Requisitos de información:** Establecen las estructuras de datos, formatos y restricciones sobre la información que el sistema debe almacenar, procesar y presentar.

Esta clasificación permite abordar de manera sistemática tanto las capacidades operativas del sistema como sus características más cualitativas, asegurando así que la

plataforma creada no sólo cumple con las funcionalidades requeridas, sino que lo haga con los estándares de calidad apropiados un entorno educativo.

2.2. Descripción funcional

A continuación, detallaremos el conjunto de operaciones y capacidades que la plataforma debe proporcionar a sus diferentes tipos de usuarios. Estas funcionalidades han sido identificadas a partir de los objetivos establecidos en la introducción y organizadas según los tres roles principales del sistema: estudiantes, profesores y administradores.

La nomenclatura que usaremos para cada uno de los requisitos será general, de forma que podrá ser utilizada a modo de referencia concreta en futuros capítulos:

- Para los requisitos funcionales, usaremos la nomenclatura **[RF X]**, siendo X el número identificador de cada requisito funcional. Adicionalmente, a partir de este momento podrá usarse la abreviación RF para referirse a Requisito Funcional.
- Para los requisitos no funcionales, usaremos la nomenclatura **[RNF Y]**, siendo Y el número identificador de cada requisito no funcional. Adicionalmente, a partir de este momento podrá usarse la abreviación RNF para referirse a Requisito No Funcional.
- Para los requisitos de información, usaremos la nomenclatura **[RI Z]**, siendo Z el número identificador de cada requisito de información. Adicionalmente, a partir de este momento podrá usarse la abreviación RI para referirse a Requisito de Información.

2.2.1. Requisitos funcionales

Gestión de Usuarios y Autenticación

[RF 1] El usuario deberá activar por primera vez su cuenta en la plataforma utilizando obligatoriamente sus credenciales institucionales. Posteriormente, podrá asociar o cambiar el correo electrónico con el que accede habitualmente, manteniendo siempre la vinculación interna con su identidad institucional.

[RF 2] El sistema debe permitir a los usuarios visualizar y editar su información personal, así como su estado dentro de la plataforma a través de un perfil de usuario único.

[RF 3] El sistema debe implementar un control de acceso basado en roles de usuario (estudiante, profesor y administrador), restringiendo funcionalidades dentro de la aplicación en base a dicho rol de usuario.

[RF 4] Los profesores podrán registrar nuevos estudiantes en sus grupos de manera individual o mediante la carga de un archivo con formato específico.

[RF 5] Los administradores podrán crear cuentas de profesores y asignarlos a grupos específicos.

Gestión de Contenidos Educativos

[RF 6] Los administradores podrán crear, editar y eliminar titulaciones, asignaturas y grupos académicos asociados, organizando el contenido de manera jerárquica (Titulación > Asignaturas > Grupos).

[RF 7] Los administradores podrán estructurar cada asignatura en temarios o unidades didácticas, estableciendo una secuencia lineal de aprendizaje. A su vez, podrán especificar el lenguaje de programación de cada asignatura o temario.

[RF 8] Los profesores tendrán a su disposición herramientas para crear ejercicios de programación, especificando el enunciado, entradas de prueba y salidas esperadas, con la posibilidad de agregar una plantilla de código inicial.

[RF 9] El sistema debe permitir asociar a cada ejercicio múltiple casos de prueba, incluyendo tanto casos públicos (visibles para estudiantes) como casos privados (ocultos para validación).

[RF 10] Cada ejercicio podrá tener especificados límites en el tiempo de ejecución, memoria máxima usada y tamaño de salida, con la posibilidad de omitir alguno a gusto del profesor.

[RF 10.1] Adicionalmente, se podrá establecer el uso de determinados tipos de datos y estructuras para resolver el problema (tanto tipos proporcionados por el propio sistema como otros definidos por el profesorado) y, cuando proceda, requisitos de eficiencia (por ejemplo, un orden de complejidad temporal esperado), que el sistema intentará inferir ejecutando el código con distintos tamaños de entrada.

[RF 11] Los profesores podrán restringir el acceso a los ejercicios por fecha o manualmente, asignando una disponibilidad temporal. Podrán definirse uno o varios períodos de entrega y, en caso de habilitarse entregas fuera del período principal, se podrán establecer penalizaciones automáticas (por ejemplo, reducción de la calificación máxima alcanzable).

Entrega y Evaluación de Ejercicios

[RF 12] Los estudiantes podrán enviar soluciones de código para los ejercicios disponibles, aceptando tantos lenguajes de programación según sea configurado.

[RF 13] Los estudiantes deberán abstraerse de la ejecución del código, es decir, es el sistema el que debe compilarlo automáticamente detectando errores de sintaxis y proporcionando mensajes de error lo más descriptivos posibles.

[RF 14] El sistema aislará la ejecución del código en un entorno independiente que garantice la seguridad del sistema anfitrión, previniendo accesos no autorizados y operaciones maliciosas.

[RF 15] Los casos de prueba asociados a un ejercicio se ejecutarán automáticamente, comparando las salidas generadas con las esperadas, previniendo también en la medida de lo posible falsos positivos.

[RF 16] El sistema debe emitir un veredicto para cada envío, clasificado como: Aceptado, Error de Compilación, Error de Ejecución, Respuesta Incorrecta, Límite de Tiempo Excedido, Límite de Memoria Excedido o Error del Sistema. Adicionalmente, el sistema podrá etiquetar internamente ciertos envíos con marcas de revisión (por ejemplo, sospecha de plagio o comportamiento anómalo) cuando los mecanismos de comparación o análisis así lo indiquen.

[RF 17] Los estudiantes tendrán acceso a una retroalimentación generada automáticamente por el sistema para cada envío, incluyendo mensajes de error, casos de prueba fallidos y métricas básicas de ejecución.

[RF 17.1] Opcionalmente, el profesorado podrá habilitar retroalimentación adicional (por ejemplo, pistas o explicaciones detalladas), pudiendo asociar a su uso una penalización configurada en la calificación máxima disponible en intentos posteriores.

[RF 18] Los estudiantes podrán, dependiendo de lo especificado por el profesor, realizar múltiples envíos para un mismo ejercicio, manteniéndose un historial completo de todos los intentos realizados. El sistema almacenará la información necesaria para comparar visualmente los distintos intentos (diferencias de código, tiempos y resultados), de forma que tanto el profesorado como el propio estudiante puedan analizar la evolución de la solución desde la idea inicial.

Seguimiento y Analítica

[RF 19] Los estudiantes podrán visualizar su progreso personal, incluyendo ejercicios completados, intentos realizados y la tasa de éxito.

[RF 20] Los estudiantes y los profesores podrán consultar el historial completo de envíos para cada ejercicio, incluyendo código enviado, veredicto y retroalimentación recibida.

[RF 21] Los profesores tendrán a su disposición herramientas para monitorizar el progreso de todos los estudiantes de su grupo, identificando tendencias y dificultades comunes, así como posibles comportamientos anómalos o plagios. Para ello, el sistema podrá emplear tanto herramientas específicas de comparación de código como indicadores derivados de la secuencia y forma de envío de los ejercicios (por ejemplo, múltiples entregas consecutivas de todos los ejercicios sin interacción previa con las pruebas).

[RF 22] El sistema tendrá herramientas para generar estadísticas agregadas para cada ejercicio, mostrando tasas de éxito, tiempo promedio de resolución y errores más frecuentes. Este punto puede estar sujeto a cambios durante el desarrollo de la plataforma.

[RF 23] Los profesores y administradores podrán exportar datos de progreso y evaluaciones en formatos estándar para análisis externos.

[RF 24] El profesorado podrá registrar evaluaciones manuales y comentarios cualitativos adicionales asociados a uno o varios envíos de un ejercicio o actividad, complementando la retroalimentación automática del sistema.

[RF 25] Los estudiantes podrán consultar, para cada ejercicio o conjunto de envíos, las calificaciones finales y los comentarios registrados manualmente por el profesorado.

[RF 26] Además de la ejecución de casos de prueba, el sistema tendrá mecanismos de comparación entre códigos y sistemas de alertas ante comportamientos sospechosos sin proporcionar retroalimentación al estudiantado ni modificar las calificaciones sin intervención docente.

2.2.2. Requisitos no funcionales

Rendimiento y Escalabilidad

[RNF 1] El sistema debe responder a las interacciones del usuario en la interfaz web (cambios de vista, navegación, acciones básicas) en un tiempo máximo de 2 segundos bajo condiciones normales de operación. Este requisito se refiere exclusivamente a la respuesta percibida en la interfaz de usuario; los procesos de evaluación intensiva podrán ejecutarse en segundo plano siempre que el usuario reciba información clara del estado de la operación.

[RNF 2] Para cada ejercicio se definirá un tiempo objetivo máximo de evaluación (compilación, ejecución de todos los casos de prueba y generación de resultados), configurable por el profesorado o la coordinación de la asignatura. Este tiempo podrá variar en función de la naturaleza del problema; por ejemplo, en ejercicios donde se evalúa la eficiencia algorítmica se podrán utilizar tamaños de datos elevados que impliquen tiempos de evaluación mayores, siempre que se informe adecuadamente al usuario.

[RNF 3] El sistema debe ser capaz de procesar simultáneamente múltiples códigos sin degradación significativa del rendimiento.

[RNF 4] El sistema debe distribuir eficientemente los recursos computacionales entre múltiples solicitudes de ejecución, evitando que un proceso monopolice todos los recursos del sistema.

Seguridad

[RNF 5] El sistema debe ejecutar todo código enviado por usuarios en entornos completamente aislados que impidan acceso al sistema de archivos del anfitrión, la red y otros recursos sensibles.

[RNF 6] El código del estudiante se ejecutará con los mínimos privilegios posibles, sin capacidad de elevar permisos o ejecutar comandos fuera del sistema.

[RNF 7] El sistema limitará CPU, memoria, procesos hijos y operaciones de E/S para cada ejecución de código.

[RNF 8] El sistema implementará mecanismos de validación y sanitización de todas las entradas de usuario, para así prevenir ataques de inyección de código.

[RNF 9] El sistema debe implementar mecanismos de autenticación robustos, utilizando protocolos seguros para el intercambio de credenciales.

[RNF 10] El sistema debe implementar un modelo de control de acceso que garantice que los usuarios solo puedan acceder a funcionalidades y datos autorizados para su rol.

[RNF 11] Todas las comunicaciones entre cliente y servidor deben realizarse mediante protocolos cifrados (HTTPS/TLS). [\[9\]](#)

[RNF 12] El sistema debe registrar en logs seguros todas las acciones administrativas críticas, como creación de usuarios, modificación de ejercicios y acceso a datos sensibles.

Usabilidad y Accesibilidad

[RNF 13] La interfaz de usuario debe ser intuitiva y requerir mínima capacitación para su uso efectivo por parte de estudiantes y profesores. [\[10\]](#)

[RNF 14] El sistema debe mantener un diseño visual consistente en todas sus páginas, utilizando patrones de interfaz reconocibles y predecibles.

[RNF 15] La interfaz web debe adaptarse correctamente a diferentes tamaños de pantalla, siendo utilizable en dispositivos de escritorio, tabletas y móviles.

[RNF 16] El sistema debe proporcionar retroalimentación visual inmediata para todas las acciones del usuario, indicando el estado de las operaciones en curso.

[RNF 26] La interfaz de usuario deberá cumplir, como mínimo, los criterios de conformidad de nivel AA de la norma WCAG 2.1 y la legislación vigente en materia de accesibilidad digital en el ámbito universitario. [\[13\]](#)

Fiabilidad y Disponibilidad

[RNF 17] El sistema debe mantener una disponibilidad mínima del 99% durante los períodos lectivos, excluyendo mantenimientos programados.

[RNF 18] El sistema debe manejar errores de forma robusta, evitando fallos catastróficos y proporcionando mensajes informativos cuando ocurran problemas.

[RNF 19] El sistema debe garantizar que la evaluación de un mismo código con los mismos casos de prueba produzca siempre el mismo resultado, asegurando justicia en la evaluación.

[RNF 20] En caso de fallo del componente de ejecución de código, el sistema debe recuperarse automáticamente y notificar al usuario sobre el problema sin perder datos de envíos.

Mantenibilidad y Portabilidad

[RNF 21] El sistema debe implementar una arquitectura modular que facilite la modificación y extensión de componentes individuales sin afectar al resto del sistema.

[RNF 22] Todo el código fuente debe estar adecuadamente documentado, incluyendo comentarios explicativos y documentación de API.

[RNF 23] El sistema *backend* debe poder ejecutarse en diferentes sistemas operativos (Linux, Windows) sin modificaciones significativas.

[RNF 27] La recarga de la página o la repetición involuntaria de la petición durante la subida del código no deberá incrementar el número de intentos contabilizados para un ejercicio. Únicamente se considerará un intento válido cuando el servidor haya recibido y aceptado el envío de forma íntegra.

Compatibilidad

[RNF 24] La interfaz web debe funcionar correctamente en las últimas versiones de los navegadores principales (Chrome, Firefox, Safari, Edge).

[RNF 25] El sistema debe aceptar código fuente en codificación UTF-8, soportando caracteres especiales y diferentes idiomas.

2.2.3. Requisitos de información

Información de Usuarios

[RI 1] El sistema debe cumplir con normativas de protección de datos (RGPD), garantizando la privacidad de la información personal de estudiantes y profesores. Esto se recoge en reglamento (UE) 2016/679 del parlamento europeo y del consejo de 27 de abril de 2016. [\[11\]](#)

[RI 2] El sistema debe mantener información sobre la pertenencia de usuarios a grupos académicos, permitiendo que un estudiante pertenezca a múltiples grupos y un profesor gestione varios grupos.

[RI 3] El sistema guardará información básica, distinguible, personal y no personal para cada usuario, pudiendo identificar a cada uno de ellos de manera independiente.

[RI 4] El sistema debe almacenar preferencias configurables por el usuario, como el estilo claro/oscuro de la aplicación.

Información de Contenidos Educativos y Seguimiento

[RI 5] El sistema guardará información básica, distinguible, para cada contenido educativo, sea ejercicio, temario o asignatura, pudiendo identificar a cada uno de ellos de manera independiente y única.

[RI 6] El sistema deberá guardar información básica y configurable o no configurable para cada restricción de ejercicio, caso de prueba, datos de envío y fechas de disponibilidad, siguiendo esta última el mismo formato a lo largo de toda la aplicación.

[RI 7] Los resultados de ejecución, compilación y retroalimentación deberán tener un formato entendible para cada usuario, siguiendo toda la misma estructura, aunque se trate de ejercicios distintos.

[RI 8] Las estadísticas de usuario y de ejercicios tendrán un formato entendible para cada usuario, pudiendo cambiarse la estructura de visualización. Sujeto a cambios.

Formatos y Restricciones

[RI 9] Los logs de actividad deben registrar eventos significativos: Tipo de evento, usuario que lo realizó, fecha y hora, detalles específicos del evento, y el tipo de usuario que se autenticó en caso de ser profesor o administrador.

[RI 10] Todos los datos de texto almacenados y procesados por el sistema deben utilizar codificación UTF-8.

[RI 11] El sistema debe aceptar archivos CSV para importación masiva de estudiantes con el siguiente formato:

- UTF-8.
- Separador: Coma (,) o punto y coma (;).
- Columnas requeridas (sujeto a cambios): DNI, Apellidos, Nombre, correo.
- Primera fila: Encabezados de columnas.
- El orden de las columnas puede variar.

[RI 12] El sistema debe aceptar archivos de código fuente con las extensiones según el lenguaje, además de que el código enviado por el estudiante no debe exceder cierto umbral a determinar.

[RI 13] Todas las fechas y horas almacenadas deben incluir zona horaria (preferentemente UTC) y seguir el formato ISO 8601. [\[12\]](#)

[RI 14] El sistema debe mantener integridad referencial entre todas las entidades relacionadas, garantizando que no existan referencias a entidades inexistentes (por ejemplo, envíos que referencien ejercicios eliminados).

2.2.4. Conclusión general

La documentación exhaustiva de estos requisitos facilita la comunicación entre todos los actores del proyecto y proporciona una referencia clara para las fases posteriores de diseño, implementación y validación. En los capítulos siguientes, se detallará cómo estos requisitos se traducen en decisiones arquitectónicas, diseños de interfaz y componentes de software específicos que materializan la visión de la plataforma educativa.

2.3. Perfil de los usuarios

Esta plataforma está naturalmente diseñada para satisfacer las necesidades de los usuarios, centrándonos en tres perfiles de usuario distintos. Cada uno de ellos tendrán sus responsabilidades, permisos y funcionalidades diferenciadas. Los perfiles interactuarán de manera complementaria para crear un ecosistema robusto, donde se equilibre la autonomía estudiantil con la supervisión docente y además una gobernanza administrativa.

Antes de profundizar en cada perfil específico, es importante establecer que todos los usuarios comparten ciertos atributos y funcionalidades comunes que formen parte del sistema base. Estos elementos transversales proporcionarán coherencia y permitirán una experiencia consistente e independiente al rol asignado:

- **Identificación única:** Cada usuario poseerá un identificador único que lo distingue en el sistema, generado automáticamente al momento del registro.
- **Autenticación centralizada:** Todos los usuarios accederán principalmente mediante el uso de credenciales, preferentemente institucionales, a través de servicios federados como *OAuth* con proveedores externos.
- **Control de acceso basado en roles:** El acceso a funcionalidades y datos se determina exclusivamente por el rol asignado, implementándose validaciones en cada petición del servidor.
- **Perfil configurable:** Independientemente del rol, cada usuario puede visualizar y modificar datos no críticos (que no sean datos personales).
- **Auditoría:** Todas las acciones significativas (consulta de datos sensibles, modificación de configuración, creación de recursos) son registradas en el servidor.

2.3.1. Estudiante

El estudiante constituye el usuario final primario de la plataforma. Su interacción se centra en el aprendizaje práctico mediante la realización de ejercicios de programación, la obtención de retroalimentación automática y el seguimiento de su progreso individual dentro del contexto de su grupo académico.

Funcionalidades

- **Acceso a ejercicios:** El estudiante accede a ejercicios de programación asignados a su grupo. [\[RF 12\]](#) [\[RF 13\]](#)
- **Entrega de soluciones:** Puede enviar múltiples soluciones para cada ejercicio. [\[RF 13\]](#) [\[RF 19\]](#) [\[RF 20\]](#)
- **Recepción de Evaluación Automática:** Tras cada envío, recibe un veredicto inmediato que clasifica la ejecución. [\[RF 17\]](#) [\[RF 18\]](#)
- **Consulta de Historial:** Accede a un historial completo de sus envíos anteriores. [\[RF 22\]](#)
- **Seguimiento de progreso:** Visualiza un panel de progreso personal en sus ejercicios completados. [\[RF 21\]](#)

Restricciones

- **No puede crear ni modificar ejercicios:** Sólo puede realizarlos según estén disponibles.
- **No puede ver código de compañeros:** Sólo el suyo propio. Queda aislado del resto.
- **Prohibición de acceso administrativo:** Nunca puede acceder a paneles administrativos.

2.3.2. Profesor

El profesor actúa como facilitador educativo y supervisor del aprendizaje estudiantil. Su rol es gestionar el acceso de estudiantes a su grupo, configurar visibilidad de ejercicios según la progresión del temario, monitorizar el progreso colectivo, identificar dificultades comunes y proporcionar retroalimentación manual cuando sea necesario.

Funcionalidades

- **Gestión de Estudiantes:** El profesor puede registrar nuevos estudiantes en su grupo de manera individual o múltiple. Además, el profesor podrá marcar estudiantes como inactivos o eliminarlos de su grupo bajo ciertas condiciones académicas (por ejemplo, baja en la asignatura), preservando en todo caso el historial de envíos y evaluaciones para fines de auditoría. [\[RF 4\]](#) [\[RI 2\]](#)
- **Análisis de evolución de los códigos enviados:** El profesor podrá visualizar, para un estudiante o un ejercicio concreto, la evolución de los distintos intentos, con el objetivo de comprender el proceso de aprendizaje. [\[RF 18\]](#) [\[RF 21\]](#)
- **Identificación de errores comunes:** El sistema facilitará al profesor acceso a reportes de errores más comunes, permitiendo detectar conceptos mal asimilados y ajustar la docencia. [\[RF 21\]](#) [\[RF 22\]](#)

- **Control de Disponibilidad de Temarios:** El profesor puede bloquear/desbloquear temarios completos para su grupo [\[RF 11\]](#)
- **Monitorización Individual:** Accede a un panel para cada estudiante. [\[RF 23\]](#)
- **Análisis Agregado del Grupo:** Visualiza estadísticas de nivel de grupo. [\[RF 22\]](#) [\[RF 23\]](#)
- **Exportación de Datos para Análisis Externo:** Puede exportar datos de progreso, evaluaciones y estadísticas. [\[RF 18\]](#)
- **Detección de posibles plagios:** El profesor dispondrá de una vista o informe que integren los resultados de las herramientas de comparación de código y de los indicadores de comportamiento sospechoso (patrones de entrega atípicos, similitud elevada, etc.), sin mostrar información directamente al estudiantado. [\[RF 21\]](#) [\[RF 24\]](#)

Restricciones

- **Gestión limitada a su grupo:** Un profesor solo ve estudiantes y datos de su grupo.
- **Limitaciones de exportación:** Solo puede exportar datos de su grupo, no datos globales.

2.3.3. Administrador

El administrador es el usuario con mayores poderes del sistema, responsable de toda la configuración, gobernanza y mantenimiento de la plataforma. Supervisa la creación y gestión de contenido educativo, gestión de usuarios (creación de cuentas de profesores), configuración técnica del sistema y aseguración de integridad y seguridad de datos.

Funcionalidades

- **Gestión de Asignaturas y Estructura Educativa:** El administrador crea y mantiene la estructura académica completa, definiendo titulaciones, las asignaturas asociadas a cada titulación y los grupos académicos vinculados a cada asignatura. [\[RF 6\]](#)
- **Creación de Ejercicios:** Desarrolla la biblioteca de ejercicios usando una interfaz de plantilla. [\[RF 8\]](#) [\[RF 9\]](#) [\[RF 10\]](#)
- **Gestión de Cuentas de Profesores:** Crea cuentas de profesores (que inicialmente no tienen asignaturas ni grupos asignados) y se encarga de su posterior vinculación a las asignaturas y grupos correspondientes. [\[RF 5\]](#)
- **Gestión de Grupos Académicos:** Los grupos académicos se definen siempre en el contexto de una asignatura concreta perteneciente a una titulación determinada. [\[RF 6\]](#)

Restricciones

- **Responsabilidad elevada:** Debe estar al tanto de peticiones de todo tipo de usuarios.
- **Auditoría completa:** Todas las acciones del administrador son registradas.

2.4. Casos de Uso

La especificación de casos de uso proporciona una perspectiva centrada en el [usuario](#) sobre cómo interactúa el sistema con sus actores principales para alcanzar objetivos específicos. Cada caso de uso representa una secuencia de interacciones que llevan a un resultado observable de valor para el usuario. Los casos de uso se especifican en tres niveles de detalle para proporcionar claridad a múltiples niveles de comprensión.

Usaremos siglas para poder referirnos de forma abreviada a los siguientes conceptos y facilitar así la redacción de este documento:

- **CU:** Caso de Uso.
- **Diagrama CU:** Diagrama de Caso de Uso.
- **CU-XY:** Caso de Uso – Número de Caso de Uso.

Las notaciones usadas en los diagramas se verán descritas en la [leyenda UML](#).

2.4.1. Diagramas de Casos de Uso

A continuación, detallaremos varios esquemas en función del perfil de usuario que tomará el papel de actor para entender cómo reaccionará el entorno con las diferentes tareas y funciones que éstos puedan hacer.

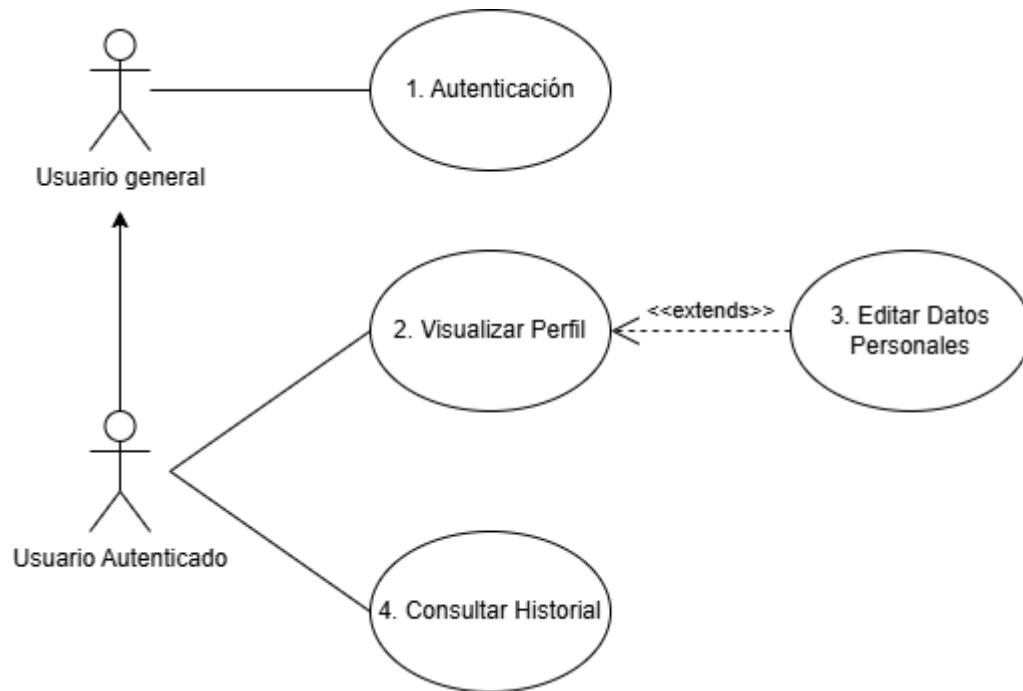


Figura 2.4.1 Casos de uso comunes (Todos los Usuarios).

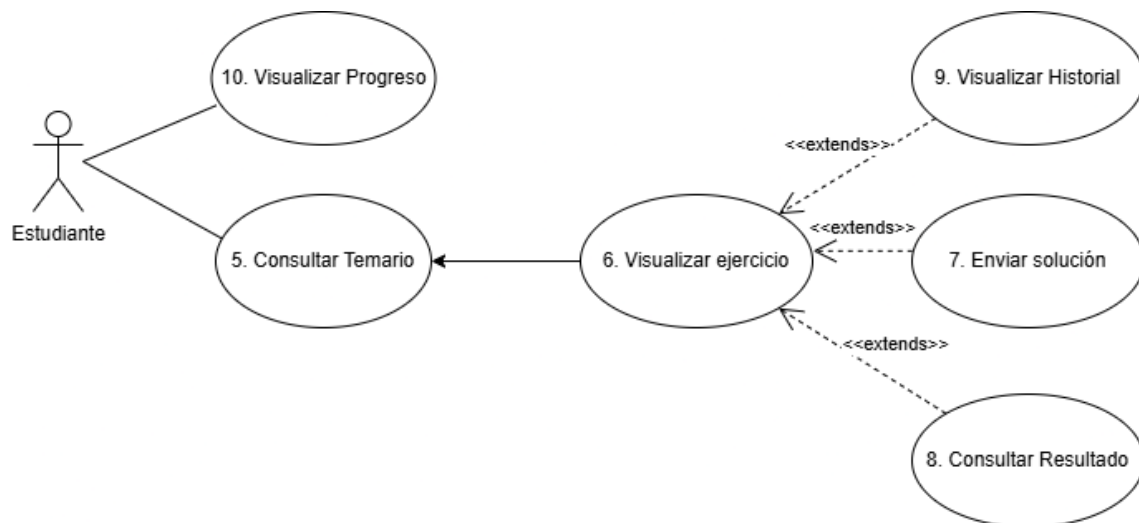


Figura 2.4.2 Casos de uso estudiantes.

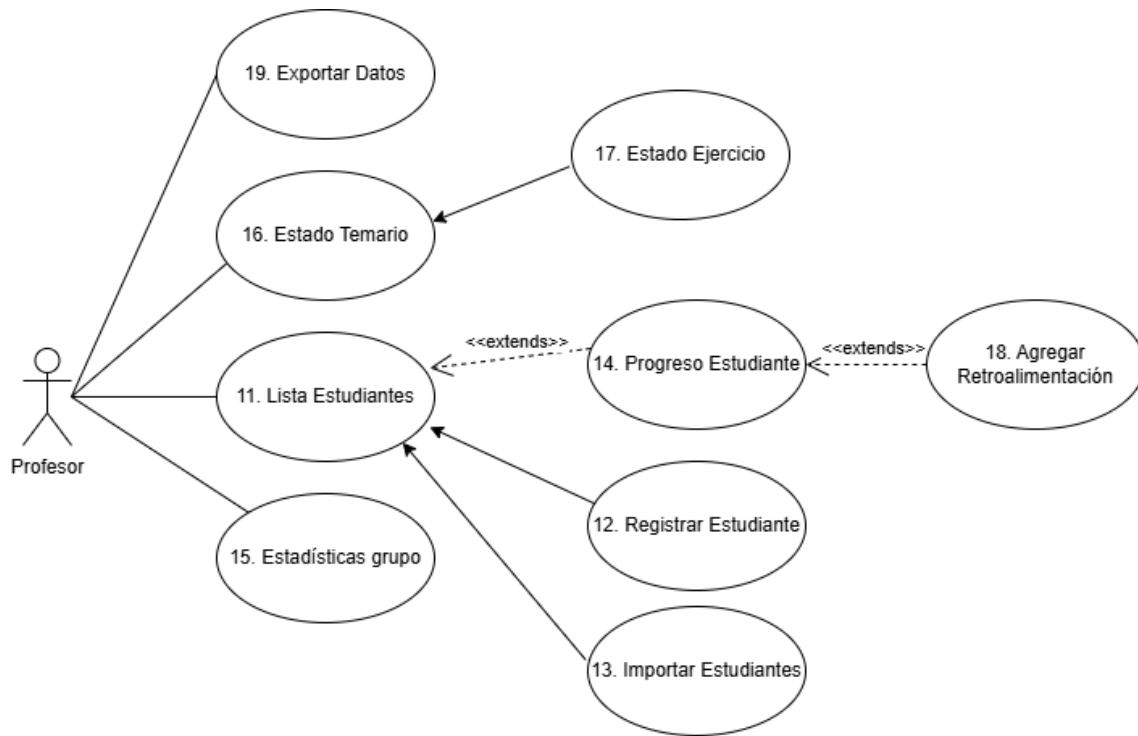


Figura 2.4.3 Casos de uso profesor.

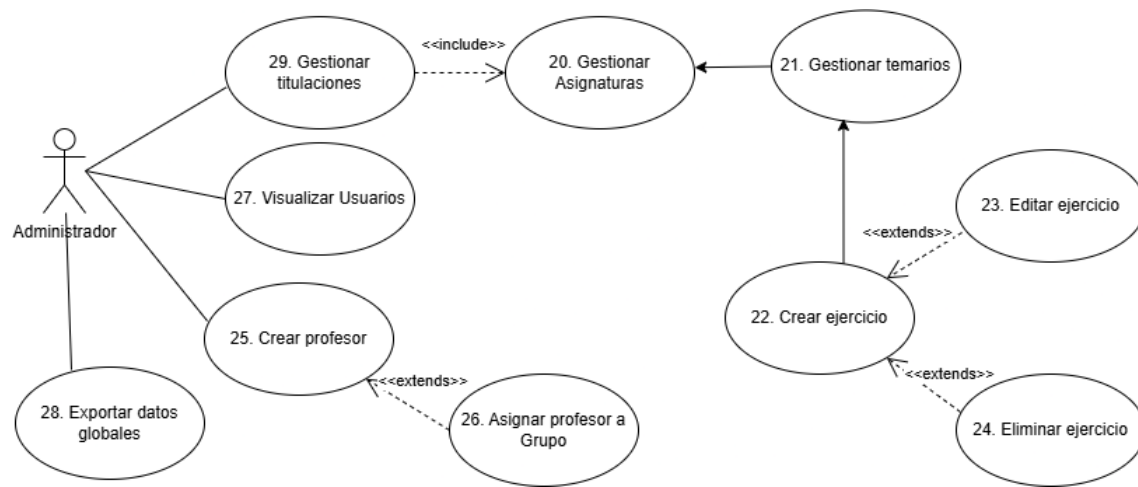


Figura 2.4.4 Casos de uso administrador.

2.4.2. Descripción de Casos de Uso

Casos de Uso Comunes (Todos los Usuarios)

ID	Caso de Uso	Actor	Dependencias
CU-01	Autenticación	Usuario General	Ninguna
CU-02	Visualizar Perfil Personal	Usuario Autenticado	CU-01
CU-03	Editar Datos Personales	Usuario Autenticado	CU-02
CU-04	Consultar Historial Actividad	Usuario Autenticado	CU-01

Tabla 2.4.1 Enumeración CU para todos los usuarios. [\[Figura 2.4.1\]](#)

CU-01	Autenticación
Actor principal	Usuario no autenticado
Dependencias	RF 1
Precondiciones	El usuario debe estar registrado en el sistema por un profesor (estudiante) o administrador (profesor).
Tipo	Primario – Esencial
Descripción	El usuario deberá autenticarse.
Secuencia	1. El usuario accede a la pantalla de inicio de sesión. 2. El sistema solicita credenciales u otro método. 3. El usuario introduce sus credenciales. 3a. Si son correctas, accede a la aplicación. 3b. Si no lo son, el sistema informa del error y vuelve al punto 2.
Postcondición	El usuario accede y puede usar las funcionalidades permitidas según su rol.
Excepciones	La cuenta no está registrada. Problemas de conexión con el proveedor de identidad. Cuenta suspendida.
Comentarios	-

CU-02	Visualizar Perfil Personal
Actor principal	Usuario autenticado
Dependencias	CU-01 , RF 2
Precondiciones	El usuario debe haberse identificado correctamente.
Tipo	Primario – Esencial
Descripción	El usuario puede visualizar la información relativa a su perfil personal, incluyendo datos y estado.
Secuencia	1. El usuario selecciona la opción de “Perfil”. 2. El sistema muestra los datos personales y el estado del perfil. 3. El usuario puede decidir modificar sus datos (ir a CU-03) o salir.
Postcondición	El usuario consulta su información personal.
Excepciones	-
Comentarios	Si el usuario opta por editar datos, pasa a CU-03.

CU-03	Editar Datos Personales
Actor principal	Usuario autenticado
Dependencias	CU-02 , RF 2

Precondiciones	El usuario debe estar visualizando su propio perfil.
Tipo	Primario – Esencial
Descripción	Permite al usuario modificar sus datos personales, como correo electrónico o preferencias.
Secuencia	1. El usuario elige editar algún campo personal. 2. El sistema solicita los nuevos valores. 3. El usuario introduce los datos. 4. El sistema valida y actualiza la información. 4a. Si hay error, se informa y vuelve al paso 2. 4b. Si es correcto, se actualizan los datos y se vuelve a CU-02.
Postcondición	Los datos personales del usuario se actualizan en el sistema.
Excepciones	Formato o datos inválidos. Campos obligatorios vacíos.
Comentarios	-

CU-04	Consultar Historial de Actividad
Actor principal	Usuario autenticado
Dependencias	CU-01
Precondiciones	El usuario debe haberse identificado correctamente.
Tipo	Primario – Esencial
Descripción	El usuario puede visualizar la información relativa a su perfil personal, incluyendo datos y estado.
Secuencia	1. El usuario selecciona la opción de “Perfil”. 2. El sistema muestra los datos personales y el estado del perfil. 3. El usuario puede decidir modificar sus datos (ir a CU-03) o salir.
Postcondición	El usuario consulta su información personal.
Excepciones	-
Comentarios	Si el usuario opta por editar datos, pasa a CU-03 .

Casos de Uso Estudiantes

ID	Caso de Uso	Actor	Dependencias
CU-05	Consultar Temarios	Estudiante	CU-01
CU-06	Visualizar Ejercicios	Estudiante	CU-05
CU-07	Enviar Solución de Código	Estudiante	CU-06
CU-08	Consultar Resultado	Estudiante	CU-07
CU-09	Consultar Historial Ejercicio	Estudiante	CU-06
CU-10	Visualizar Progreso	Estudiante	CU-01

Tabla 2.4.2 Enumeración CU para todos los usuarios. [\[Figura 2.4.2\]](#)

CU-05	Consultar Temarios
Actor principal	Estudiante
Dependencias	CU-01 , RF 12 , RF 13
Precondiciones	El estudiante debe haber iniciado sesión correctamente.
Tipo	Primario – Esencial
Descripción	El estudiante puede consultar la lista de temarios disponibles en su grupo.
Secuencia	1. El estudiante accede al apartado de “Temario”. 2. El sistema presenta la lista jerárquica de temarios disponibles para su grupo.

	3. El estudiante selecciona el temario que desea explorar.
Postcondición	El estudiante visualiza el contenido y la estructura del temario. Puede seleccionar ejercicios asociados.
Excepciones	No hay temarios disponibles para el grupo. Error de conexión. Permiso denegado por estado del grupo.
Comentarios	El acceso a ciertos temarios depende de la configuración establecida por el profesor.

CU-06	Visualizar Ejercicio
Actor principal	Estudiante
Dependencias	CU-05 , RF 13 , RF 14
Precondiciones	El temario debe estar desbloqueado y disponible. El estudiante debe haber seleccionado un tema.
Tipo	Primario – Esencial
Descripción	El estudiante visualiza el enunciado de un ejercicio concreto, sus restricciones y todo lo disponible.
Secuencia	1. El estudiante selecciona un ejercicio desde el temario. 2. El sistema muestra el enunciado, ejemplos y opciones disponibles. 3. El estudiante puede decidir enviar solución, ver historial o volver.
Postcondición	El estudiante entiende el ejercicio y lo puede resolver o consultar su historial.
Excepciones	El ejercicio no está disponible. Error al cargar los datos. Permiso denegado.
Comentarios	El ejercicio puede estar acompañado de pistas, temporizador y otros elementos configurables.

CU-07	Enviar Solución de Código
Actor principal	Estudiante
Dependencias	CU-06 , RF 13 , RF 14
Precondiciones	El estudiante debe estar visualizando un ejercicio disponible y no haber superado el límite de envíos si aplica.
Tipo	Primario – Esencial
Descripción	El estudiante introduce su código (editándolo o subiéndolo) y lo envía para su evaluación automática.
Secuencia	1. El estudiante accede a la interfaz de envío. 2. Introduce o carga su solución. 3. Selecciona el lenguaje si es configurable. 4. Hace clic en “Enviar”. 5. El sistema valida, almacena y lanza la evaluación. 6. Se notifica que la solución está en proceso de evaluación.
Postcondición	La solución queda registrada como un envío y comienza el proceso de evaluación. El estudiante pasa a consultar el resultado (CU-08).
Excepciones	Código vacío o formato no válido. Archivos demasiado grandes o con lenguaje no permitido. Límite de envíos alcanzado. Error del sistema.

Comentarios	El tiempo de evaluación, mensajes de estado y restricciones son comunicados de forma transparente.
-------------	--

CU-08	Consultar Resultado
Actor principal	Estudiante
Dependencias	CU-07 , RF 17 , RF 18
Precondiciones	Haber realizado un envío recientemente. El sistema debe haber procesado la evaluación.
Tipo	Primario – Esencial
Descripción	El estudiante consulta el veredicto de su envío junto con la retroalimentación detallada.
Secuencia	1. El estudiante accede al resumen tras el envío. 2. El sistema presenta el veredicto, mensajes de error (si los hay), casos de prueba superados o fallidos, estadísticas del envío. 3. El estudiante puede decidir consultar su historial, intentar de nuevo u optimizar el código.
Postcondición	El estudiante conoce el resultado de su envío y obtiene información útil para mejorar.
Excepciones	Error de comunicación con el evaluador. El resultado aún no está disponible por retraso. Resultado inconcluso (se informa al usuario).
Comentarios	La retroalimentación se presenta de manera clara, destacando errores específicos y sugerencias de mejora.

CU-09	Visualizar Historial de Ejercicio
Actor principal	Estudiante
Dependencias	CU-06 , RF 22
Precondiciones	El estudiante debe estar viendo un ejercicio que ya haya intentado previamente.
Tipo	Secundario – Esencial
Descripción	El estudiante consulta el historial de envíos realizados a ese ejercicio, comparando versiones, resultados y fechas de intento.
Secuencia	1. El estudiante selecciona la opción de ver historial. 2. El sistema muestra una lista de todos los envíos previos. 3. El estudiante puede comparar dos versiones o ver detalles de la retroalimentación.
Postcondición	El estudiante conoce el resultado de su envío y obtiene información útil para mejorar.
Excepciones	No existen envíos previos para el ejercicio. Error al recuperar datos del historial.
Comentarios	Puede incluir herramientas para filtrado, comparación de código o descarga del historial.

CU-10	Visualizar Progreso
Actor principal	Estudiante
Dependencias	CU-01 , RF 21 , RI 14
Precondiciones	El estudiante ha realizado al menos un envío en la plataforma.

Tipo	Primario – Esencial
Descripción	Permite al estudiante visualizar un panel resumen de su rendimiento: ejercicios completados, porcentaje de avance, tasa de éxito, racha de días activos, etc.
Secuencia	1. El estudiante accede a la sección de progreso personal. 2. El sistema muestra un panel con las estadísticas, tendencias y posibles recomendaciones de mejora. 3. El estudiante puede filtrar por temario, por fecha o por ejercicio.
Postcondición	El estudiante obtiene una visión clara de su evolución y motivación para seguir avanzando.
Excepciones	No existen datos suficientes para mostrar métricas. Error al agregar datos estadísticos.
Comentarios	Este panel fomenta la autogestión y el aprendizaje autónomo.

Casos de Uso Profesores

ID	Caso de Uso	Actor	Dependencias
CU-11	Consultar Lista Estudiantes	Profesor	CU-01
CU-12	Registrar Estudiante Único	Profesor	CU-11
CU-13	Importar Estudiantes	Profesor	CU-11
CU-14	Consultar Progreso Estudiante	Profesor	CU-11
CU-15	Analizar Estadísticas Grupo	Profesor	CU-01
CU-16	Cambiar Estado Temario	Profesor	CU-01
CU-17	Cambiar Estado Ejercicio	Profesor	CU-16
CU-18	Añadir Notas a Envío	Profesor	CU-14
CU-19	Exportar Datos de Grupo	Profesor	CU-01

Tabla 2.4.3 Enumeración CU para todos los profesores. [\[Figura 2.4.3\]](#)

CU-11	Consultar Lista de Estudiantes
Actor principal	Profesor
Dependencias	CU-01 , RF 4 , RI 2
Precondiciones	El profesor debe haber iniciado sesión correctamente y tener un grupo asignado.
Tipo	Primario – Esencial
Descripción	El profesor accede a la lista de estudiantes asignados a su grupo, visualizando información básica y estado de la cuenta.
Secuencia	1. El profesor entra en la sección “Estudiantes”. 2. El sistema muestra la lista actualizada de estudiantes del grupo. 3. Puede ordenar/filtrar e iniciar otras acciones.
Postcondición	El profesor obtiene visualización completa y puede iniciar acciones relacionadas.
Excepciones	El grupo no tiene estudiantes. Error en la recuperación de datos. Permisos insuficientes.
Comentarios	—

CU-12	Registrar Estudiante Individual
Actor principal	Profesor
Dependencias	CU-11 , RF 4 , RI 1

Precondiciones	El profesor debe estar viendo la lista del grupo y el correo/DNI del estudiante no debe estar registrado.
Tipo	Secundario – Esencial
Descripción	El profesor registra manualmente un nuevo estudiante en el grupo, completando el formulario con datos básicos.
Secuencia	1. El profesor selecciona la opción “Registrar estudiante”. 2. Rellena los campos requeridos. 3. El sistema valida datos. 4a. Si son válidos, registra al estudiante en la BD. 4b. Si no, informa del error y repite el proceso.
Postcondición	El nuevo estudiante queda añadido al grupo y aparece en la lista principal.
Excepciones	Correo o DNI duplicados. Formato inválido en campos. Campos obligatorios vacíos.
Comentarios	—

CU-13	Importar Estudiantes
Actor principal	Profesor
Dependencias	CU-11 , RF 4 , RI 1
Precondiciones	El profesor debe estar viendo la lista de su grupo. Debe disponer de un archivo CSV con el formato correcto.
Tipo	Secundario – Esencial
Descripción	Permite al profesor importar varios estudiantes de golpe cargando un archivo CSV con los datos requeridos.
Secuencia	1. El profesor pulsa “Importar estudiantes”. 2. Selecciona archivo. 3. El sistema valida y carga. 4a. Si hay errores, muestra detalle y pide corrección. 4b. Si todo es correcto, añade todos los estudiantes y actualiza la lista.
Postcondición	Todos los estudiantes válidos quedan añadidos al grupo y visibles en la lista principal.
Excepciones	Archivo no compatible o mal formado. Errores en datos del archivo. Límites superados (cantidad máxima en grupo).
Comentarios	Se informa claramente de qué filas dieron error para corrección rápida.

CU-14	Progreso Estudiante
Actor principal	Profesor
Dependencias	CU-11 , RF 6 , RI 14
Precondiciones	El profesor debe seleccionar a un estudiante concreto y este debe tener actividad registrada.
Tipo	Secundario – Esencial
Descripción	Permite al profesor visualizar el resumen del progreso de un estudiante: ejercicios completados, éxito, racha, avances, errores frecuentes.
Secuencia	1. El profesor selecciona “Ver progreso” en un estudiante. 2. El sistema muestra panel detallado de actividad, avance y éxito. 3. Puede optar a agregar retroalimentación.
Postcondición	El profesor conoce el rendimiento del estudiante para orientar su seguimiento.

Excepciones	No hay datos de progreso para mostrar. Error de recuperación de datos.
Comentarios	—

CU-15	Estadísticas de Grupo
Actor principal	Profesor
Dependencias	CU-01 , RF 23 , RF 22
Precondiciones	El profesor debe tener un grupo asignado y datos de actividad registrados.
Tipo	Primario – Esencial
Descripción	El profesor accede a estadísticas agregadas de todo el grupo: ejercicios completados, tasas de éxito, errores frecuentes y evolución global.
Secuencia	1. El profesor entra en “Estadísticas grupo”. 2. El sistema muestra los indicadores clave y gráficas con evolución. 3. Puede filtrar por temario, periodo, dificultad.
Postcondición	El profesor puede identificar áreas donde el grupo tiene dificultades y ajustar la enseñanza.
Excepciones	No hay datos suficientes. Error en procesamiento de estadísticas. Permisos insuficientes.
Comentarios	Ayuda a planificar la docencia y anticipar problemas.

CU-16	Estado Temario
Actor principal	Profesor
Dependencias	CU-01 , RF 11
Precondiciones	El profesor debe tener temarios en el sistema y permisos sobre su grupo.
Tipo	Primario – Esencial
Descripción	Permite al profesor consultar y cambiar el estado (bloqueado/desbloqueado) de los temarios disponibles para su grupo.
Secuencia	1. El profesor accede a “Estado temario”. 2. El sistema muestra la lista de temarios con su estado actual. 3. El profesor puede bloquear/desbloquear el temario seleccionado y confirmar el cambio.
Postcondición	Se actualiza la disponibilidad de temarios para el grupo y cambia el acceso de los estudiantes.
Excepciones	No existen temarios. Error al modificar el estado. Permiso denegado para el grupo.
Comentarios	Ayuda a planificar la docencia y anticipar problemas.

CU-17	Estado Ejercicio
Actor principal	Profesor
Dependencias	CU-16 , RF 11
Precondiciones	El temario debe estar desbloqueado y el profesor debe tener permisos de gestión sobre el grupo.
Tipo	Secundario – Esencial
Descripción	Permite al profesor definir fechas de disponibilidad (inicio/fin) para ejercicios individuales dentro de un temario desbloqueado.

Secuencia	1. El profesor accede a la sección de “Estado temario” y selecciona un temario desbloqueado. 2. El sistema muestra la lista de ejercicios asociados a ese temario con su estado actual de disponibilidad. 3. El profesor selecciona un ejercicio concreto y elige entre activarlo, desactivarlo o modificar sus fechas de inicio y fin. 4. El sistema solicita confirmación de los cambios. 5. El profesor confirma y el sistema valida los datos introducidos. 6. El sistema actualiza la disponibilidad del ejercicio y muestra el nuevo estado.
Postcondición	La disponibilidad del ejercicio seleccionado queda actualizada para el grupo, de acuerdo con las fechas y el estado configurados.
Excepciones	No existen ejercicios asociados al temario. Los datos introducidos (por ejemplo, fechas) no son válidos. Se produce un error al actualizar la información. El profesor no tiene permisos suficientes sobre el grupo.
Comentarios	Cambios reflejados en tiempo real para los estudiantes.

CU-18	Agregar retroalimentación
Actor principal	Profesor
Dependencias	CU-14 , RF 18
Precondiciones	El profesor debe estar viendo el progreso de un estudiante concreto y haber identificado algún envío para comentar.
Tipo	Secundario – Esencial
Descripción	El profesor puede añadir comentarios o sugerencias a los envíos específicos de un estudiante, además del generado automáticamente.
Secuencia	1. El profesor accede a un envío concreto dentro del progreso del estudiante. 2. Escribe la retroalimentación o comentario. 3. El sistema almacena y muestra la retroalimentación en el perfil del estudiante.
Postcondición	Se añade retroalimentación al envío y el estudiante puede consultarla junto al resultado.
Excepciones	Error al guardar el comentario. Permisos insuficientes. No existe el envío.
Comentarios	Ideal para corregir errores frecuentes o orientar la mejora del estudiante.

CU-19	Exportar Datos
Actor principal	Profesor
Dependencias	CU-01 , RF 22
Precondiciones	El profesor debe tener actividad de grupo registrada y permisos para exportación de datos.
Tipo	Primario – Esencial
Descripción	Permite al profesor exportar los datos de progreso, estadísticas y actividad del grupo en formatos estándar.
Secuencia	1. El profesor selecciona “Exportar datos”. 2. El sistema solicita formato de exportación. 3. El profesor confirma. 4. El sistema genera el archivo y se lo presenta para descarga.

Postcondición	El profesor dispone del archivo y puede analizar los datos externamente.
Excepciones	No existen datos exportables. Error en la generación de archivo. Permisos insuficientes.
Comentarios	Compatible para manejo en Excel, hojas de cálculo y herramientas de reporte académico.

Casos de Uso Administradores

ID	Caso de Uso	Actor	Dependencias
CU-20	Gestionar asignaturas	Administrador	CU-01
CU-21	Gestionar temarios	Administrador	CU-20
CU-22	Crear Ejercicio	Administrador	CU-21
CU-23	Editar Ejercicio	Administrador	CU-22
CU-24	Eliminar Ejercicio	Administrador	CU-22
CU-25	Crear Cuenta de Profesor	Administrador	CU-01
CU-26	Asignar Profesor a Grupo	Administrador	CU-25
CU-27	Visualizar Usuarios	Administrador	CU-01
CU-28	Exportar Datos Globales	Administrador	CU-01
CU-29	Gestionar Titulaciones	Administrador	CU-01

Tabla 2.4.4 Enumeración CU para todos los administradores. [\[Figura 2.4.4\]](#)

CU-20	Gestionar Asignaturas
Actor principal	Administrador
Dependencias	CU-01 , RF 6 , RI 4
Precondiciones	El administrador debe haber iniciado sesión correctamente.
Tipo	Primario – Esencial
Descripción	El administrador puede crear, editar, visualizar y archivar asignaturas en el sistema.
Secuencia	1. El administrador accede a la sección "Asignaturas". 2. El sistema muestra la lista de asignaturas existentes. 3. El administrador puede seleccionar crear nueva o editar una existente. 4a. Crea nueva: especifica código, nombre, descripción, curso académico. 4b. Edita: modifica datos o archiva la asignatura. 5. El sistema valida y actualiza.
Postcondición	Las asignaturas quedan registradas en el sistema y disponibles para crear temarios.
Excepciones	Datos inválidos o incompletos. Código de asignatura duplicado. Error en la actualización.
Comentarios	Las asignaturas pueden contener múltiples temarios organizados jerárquicamente.

CU-21	Gestionar Temarios
Actor principal	Administrador
Dependencias	CU-20 , RF 7 , RI 5
Precondiciones	Debe existir al menos una asignatura creada en el sistema.
Tipo	Primario – Esencial
Descripción	El administrador puede crear, editar y gestionar temarios dentro de asignaturas existentes.

Secuencia	1. El administrador selecciona una asignatura y accede a "Temarios". 2. Visualiza la lista de temarios e información asociada. 3. Puede crear nuevo temario o editar existente. 4a. Crea: especifica título, descripción, orden y conceptos clave. 4b. Edita: modifica datos o reordena temarios. 5. El sistema valida y actualiza.
Postcondición	Los temarios quedan registrados en la asignatura y disponibles para agregar ejercicios.
Excepciones	Datos inválidos. Asignatura padre no existe. Error en la actualización. Orden inválido.
Comentarios	Los temarios forman la estructura central de la organización educativa.

CU-22	Crear Ejercicio
Actor principal	Administrador
Dependencias	CU-21 , RF 8 , RF 9 , RF 10 , RI 6 , RI 7 , RI 8
Precondiciones	Debe existir un temario en el sistema. El administrador debe tener acceso a las herramientas de edición.
Tipo	Primario – Esencial
Descripción	El administrador crea nuevos ejercicios especificando enunciado, casos de prueba, restricciones y solución de referencia.
Secuencia	1. El administrador accede a "Crear ejercicio". 2. Rellena los datos generales (título, enunciado, temario, dificultad). 3. Especifica restricciones (tiempo, memoria, tamaño salida, procesos). 4. Define casos de prueba (públicos/privados) e introduce solución de referencia. 5. El sistema compila y valida solución contra casos. 6a. Si la solución pasa todos los casos, confirma creación. 6b. Si falla, solicita corrección.
Postcondición	El ejercicio queda registrado en el sistema y disponible para que profesores lo desbloqueen.
Excepciones	Enunciado sin ejemplos. Solución falla casos de prueba. Restricciones inválidas. Temario no existe. Datos incompletos.
Comentarios	La validación de solución es crítica para garantizar la calidad pedagógica del ejercicio.

CU-23	Editar Ejercicio
Actor principal	Administrador
Dependencias	CU-22 , RF 8 , RI 6 , RI 7 , RI 8
Precondiciones	Debe existir un ejercicio previo en el sistema.
Tipo	Secundario – Esencial
Descripción	El administrador modifica campos de un ejercicio existente: enunciado, casos de prueba, restricciones o solución de referencia.
	1. El administrador accede a la lista de ejercicios y selecciona uno. 2. Accede a modo edición.

Secuencia	3. Modifica los campos requeridos. 4. Si modifica solución, el sistema revalida contra todos los casos. 5a. Si todo es correcto, guarda cambios. 5b. Si hay errores, muestra problemas y pide corrección.
Postcondición	El ejercicio se actualiza y los cambios son visibles para profesores y estudiantes (si estaba disponible)
Excepciones	Datos inválidos. Solución de referencia falla validación. Ejercicio no existe. Error en guardar cambios.
Comentarios	Útil para corregir errores o mejorar ejercicios tras identificar problemas en el uso real.

CU-24	Eliminar Ejercicio
Actor principal	Administrador
Dependencias	CU-22 , RF 8
Precondiciones	Debe existir un ejercicio en el sistema.
Tipo	Secundario – Esencial
Descripción	El administrador archiva (no elimina físicamente) un ejercicio, preservando datos históricos de envíos previos.
Secuencia	1. El administrador selecciona ejercicio a eliminar. 2. El sistema solicita confirmación. 3a. Si confirma, marca el ejercicio como archivado. 3b. Los datos de envíos y resultados se preservan. 3c. El ejercicio deja de ser visible para nuevos intentos.
Postcondición	El ejercicio se archiva, los estudiantes no pueden acceder a él, pero su historial se mantiene.
Excepciones	Ejercicio no existe. Error al archivar. Permisos insuficientes.
Comentarios	La preservación de datos históricos es crítica para evaluaciones y auditoría.

CU-25	Crear Cuenta de Profesor
Actor principal	Administrador
Dependencias	CU-01 , RF 5
Precondiciones	El administrador debe haber iniciado sesión correctamente y disponer de permisos para crear nuevas cuentas de usuario de tipo profesor.
Tipo	Primario – Esencial
Descripción	El administrador debe tener permisos para crear cuentas de usuario.
Secuencia	1. El administrador accede a "Crear profesor". 2. Rellena datos básicos (nombre, apellidos, correo institucional, DNI). 3. El sistema valida datos e integridad (correo único, formato válido). 4. El sistema genera credenciales temporales o envía invitación de activación. 5. El profesor queda registrado, pero sin grupos asignados aún.
Postcondición	La cuenta de profesor está creada y puede ser asignada a grupos mediante CU-26 .
Excepciones	Correo o DNI duplicados. Formato inválido en campos.

	Campos obligatorios vacíos. Error al crear cuenta.
Comentarios	El profesor no accede a grupos hasta que se asigne explícitamente mediante CU-26 .

CU-26	Asignar Profesor a Grupo
Actor principal	Administrador
Dependencias	CU-25 , RF 5
Precondiciones	Debe existir al menos un profesor y un grupo en el sistema.
Tipo	Secundario – Esencial
Descripción	El administrador vincula un profesor existente con uno o varios grupos académicos.
Secuencia	1. El administrador accede a "Asignar profesor". 2. Selecciona un profesor de la lista. 3. Selecciona grupo(s) a asignar. 4. Confirma la asignación. 5. El sistema vincula al profesor con el grupo y actualiza permisos.
Postcondición	El profesor accede a los grupos asignados y puede gestionar estudiantes en ellos.
Excepciones	Profesor o grupo no existen. Profesor ya asignado al grupo. Error en la asignación.
Comentarios	Un profesor puede estar asignado a múltiples grupos; cambios son inmediatos.

CU-27	Visualizar Usuarios
Actor principal	Administrador
Dependencias	CU-01 , RI 1
Precondiciones	Debe haber usuarios registrados en el sistema.
Tipo	Primario – Esencial
Descripción	El administrador accede a una vista global de todos los usuarios del sistema.
Secuencia	1. El administrador entra en "Usuarios". 2. El sistema muestra lista filtrable por tipo de usuario, grupo, estado. 3. Puede buscar por nombre, correo o DNI. 4. Visualiza detalles de cada usuario (datos, actividad, estado de cuenta).
Postcondición	El administrador obtiene una visión global del sistema para supervisión y auditoría.
Excepciones	No existen usuarios registrados. Error en la recuperación de datos. Permisos insuficientes.
Comentarios	Herramienta crítica para auditoría y supervisión del sistema.

CU-28	Exportar Datos Globales
Actor principal	Administrador
Dependencias	CU-01 , RF 23

Precondiciones	Debe haber datos registrados en el sistema y permisos de exportación global.
Tipo	Primario – Esencial
Descripción	El administrador exporta datos completos de la plataforma (usuarios, ejercicios, resultados, estadísticas) en múltiples formatos.
Secuencia	1. El administrador selecciona "Exportar datos globales". 2. Elige parámetros. 3. Confirma exportación. 4. El sistema genera archivo comprimido y lo presenta para descarga.
Postcondición	El administrador dispone de un archivo completo para análisis, reportes o migración de datos.
Excepciones	No existen datos exportables. Error en la generación de archivo. Tamaño de exportación demasiado grande.
Comentarios	Herramienta de vital importancia para auditoría institucional y análisis de desempeño educativo.

CU-29	Gestionar Titulaciones
Actor principal	Administrador
Dependencias	CU-01 , RF 6 , RI 5
Precondiciones	El administrador debe haber iniciado sesión correctamente.
Tipo	Primario – Esencial
Descripción	El administrador puede crear, editar, visualizar y archivar titulaciones en el sistema, definiendo para cada una de ellas los metadatos básicos que permitirán asociar asignaturas y grupos.
Secuencia	1. El administrador accede a la sección «Titulaciones». 2. El sistema muestra la lista de titulaciones existentes, con sus datos básicos. 3. El administrador puede seleccionar crear una nueva titulación o editar/archivar una existente. 3a. En la creación, introduce código de titulación, nombre, descripción y otros metadatos relevantes (por ejemplo, centro, plan de estudios). 3b. En la edición, modifica los datos o marca la titulación como archivada si ya no está vigente. 4. El sistema valida los datos y actualiza la información correspondiente.
Postcondición	Las titulaciones quedan registradas o actualizadas en el sistema y pueden servir como contenedor para las asignaturas asociadas.
Excepciones	Código de titulación duplicado; datos inválidos o incompletos; error en la actualización.
Comentarios	-

Tipo de línea	Nombre	Significado	Notación
Sólida	Asociación	Actor interactúa directamente con CU.	–
Sólida con flecha	Prerrequisito	CU accesible cuando se cumpla anterior.	→
Punteada	<<extend>>	CU es funcionalidad adicional.	..
Punteada	<<include>>	CU implica o requiere a otro posterior.	..

Tabla 2.4.5 Leyenda de Relaciones UML.

2.5. Criterios de validación

La validación de los requisitos y casos de uso especificados en este capítulo constituye un elemento fundamental para asegurar que el sistema desarrollado responde adecuadamente a las necesidades identificadas. Los criterios de validación permiten verificar que cada requisito es correcto, completo y satisface las expectativas de los usuarios.

A lo largo del desarrollo de la aplicación, se implementarán pruebas de software al finalizar cada parte que desarrollemos. Estas pruebas abarcarán desde pruebas unitarias y de integración, destinadas a verificar el funcionamiento integral del código, hasta pruebas propias de validación que aseguren el cumplimiento de los requisitos previamente descritos. Este enfoque de evaluación no solo garantiza la integridad técnica del proyecto, sino que también se alinea con la adhesión a los estándares y especificaciones establecidos desde la fase inicial del proyecto.

Los requisitos especificados en este capítulo han sido cuidadosamente diseñados para alinearse con los objetivos definidos en la introducción, las restricciones técnicas identificadas y las necesidades de los distintos [perfiles de usuario](#). La estructura de validación establecida proporciona confianza en que el sistema desarrollado será funcional, seguro y satisfará las expectativas de la comunidad académica. Durante las fases posteriores de diseño e implementación, estos criterios de validación servirán como referencia para verificar que el desarrollo avanza conforme a lo especificado.

Aunque el presente documento es el resultado del análisis técnico, la validación completa de los requisitos requiere confirmación de los *stakeholders* (actores) principales:

Estudiantes: Validarán que los casos de uso ([CU-05](#) a [CU-10](#)) cubren adecuadamente sus necesidades de aprendizaje y que la experiencia de usuario descrita es intuitiva y motivante.

Profesores: Confirmarán que los casos de uso ([CU-11](#) a [CU-19](#)) proporcionan herramientas suficientes para gestionar sus grupos, monitorizar el progreso y ajustar la docencia conforme a necesidades identificadas.

Administrador/Departamento: Validará que los requisitos de seguridad, integridad de datos y auditoría son suficientes para garantizar el cumplimiento normativo y la operación sostenible del sistema.

Nota: Tanto yo, autor del TFM, como el tutor de este mismo proyecto tomarán los roles correspondientes anteriormente descritos, asegurándose de tener en cuenta tantos casos como sea posible.

3. Plan de Proyecto

Antes de iniciar las fases de diseño e implementación, realizaremos una planificación rigurosa que guíe el desarrollo de la plataforma. En el capítulo anterior se analizaron exhaustivamente los requisitos de software; funcionales, no funcionales y de información, junto con los casos de uso que definen la interacción entre los distintos actores y del sistema. El presente capítulo aborda la planificación integral del proyecto, estableciendo los cimientos organizativos, temporales y económicos sobre los que se sustentará esta plataforma.

Este capítulo se estructura en cuatro secciones fundamentales. En primer lugar, se detalla la organización y los recursos necesarios para llevar a cabo el proyecto, diferenciando entre recursos humanos —con sus roles, competencias y costes asociados— y recursos materiales —tanto hardware como software—. En segundo lugar, se desarrolla una planificación temporal exhaustiva basada en estimaciones de tiempo para cada fase y sprint del desarrollo, materializada mediante diagramas de Gantt que permiten visualizar las dependencias entre tareas. En tercer lugar, se presenta un estudio de las tecnologías candidatas y seleccionadas para la implementación del sistema, justificando cada elección en función de los requisitos identificados. Finalmente, se elabora una estimación de costes que integra todos los factores anteriores para determinar el presupuesto total del proyecto.

La metodología de desarrollo adoptada será ágil, basada en *sprints*, lo que permitirá entregar funcionalidades completas y operativas de forma incremental. Esta aproximación facilita la validación continua con el cliente —rol que asumirá el tutor del presente trabajo— y posibilita ajustes tempranos ante cambios en los requisitos o dificultades técnicas imprevistas. Cada sprint culminará con una versión funcional del sistema que incorpore los requisitos priorizados para dicho período.

3.1. Organización y recursos

Para desarrollar una plataforma de la complejidad que presenta este proyecto —que integra un sistema de gestión de contenidos educativos, un módulo de ejecución segura de código y una interfaz adaptada a múltiples roles de usuario—, es fundamental planificar una organización exhaustiva que contemple tanto los recursos humanos como los materiales necesarios.

Se adoptará una metodología de desarrollo ágil basada en sprints que permitirá entregar partes funcionales del proyecto de forma continua e iterativa. Esta aproximación posibilita realizar entregas periódicas al cliente, cuyo rol asumirá el propio tutor de este Trabajo Fin de Máster, mientras que el equipo de desarrollo será ejercido por el autor de este. La retroalimentación obtenida tras cada sprint guiará las decisiones de diseño e implementación de las iteraciones subsiguientes.

Por otra parte, deben considerarse factores como el requerimiento de recursos, entre los que se diferencian los recursos humanos y los recursos materiales. A continuación, se detallan y analizan estos recursos, estableciendo las bases para la estimación de costes que se desarrollará posteriormente.

3.1.1. Recursos Humanos

Todos los roles que se describen a continuación serán ejercidos por el autor de este Trabajo Fin de Máster. No obstante, para realizar una estimación de costes realista y profesional, se consideran los distintos perfiles técnicos que intervendrían en un proyecto de esta naturaleza en un entorno empresarial, con los salarios medios del sector informático en España. [14]

La complejidad del proyecto (que abarca desde el diseño de arquitecturas de microservicios hasta la implementación de entornos de ejecución aislada de código) requiere la participación de perfiles con competencias específicas y diferenciadas:

Roles	Salario Anual Bruto (€)	Salario por hora (€)
Analista	31.500	16,15
Administrador de Base de Datos (ABD)	30.000	15,38
Ingeniero de Requisitos (IR)	35.000	17,95
Ingeniero de Software (IS)	35.000	17,95
Programador	28.500	14,62
Ingeniero de Seguridad (ISE)	38.000	19,49

Tabla 3.1. Salario medio del personal técnico.

Los valores expuestos en la [Tabla 3.1](#) son aproximados y se han obtenido de referencias salariales del sector tecnológico español [14]. Pueden experimentar variaciones en función del momento de consulta y la ubicación geográfica.

Para cada sprint, los diferentes roles deberán repartirse las tareas o secciones que vayan surgiendo. Estas tareas se dividirán a su vez según la capa del proyecto a la que pertenezcan: **frontend** (interfaz de usuario), **backend** (lógica de negocio y API) o **capa de ejecución** (entorno aislado para evaluación de código). Cada rol participará activa o pasivamente en cada tarea del sprint, aunque será una persona la asignada directamente como responsable.

Roles	Salario Anual Bruto (€)	Salario por hora (€)
Diseño de la base de datos	Backend	ABD
Diseño de la lógica de la aplicación	Backend	IS
Diseño de la lógica de la vista	Frontend	IS
Diseño del módulo de ejecución	Capa de Ejecución	ISE
Implementación del diseño	Todas las capas	Programador
Pruebas unitarias y de integración	Todas las capas	IS
Pruebas de seguridad	Capa de ejecución	ISE
Documentación	Todas las capas	Programador

Tabla 3.2. Secciones generales de cada sprint.

Teniendo en cuenta esta información y que se seguirá el patrón arquitectónico cliente-servidor con separación en microservicios, se calculará la estimación del coste del capital humano —así como su división en tareas— en el apartado correspondiente de estimación de costes.

3.1.2. Recursos Materiales

Para la realización de este trabajo se necesitarán tanto equipos informáticos como recursos software con los que poder diseñar, desarrollar y probar la implementación de la plataforma. La amortización de los recursos tanto hardware como software se calculará según lo regulado por la Ley 27/2014 del 27 de noviembre sobre el Impuesto sobre Sociedades, publicada en el Boletín Oficial del Estado a fecha de 28 de noviembre de 2014. [\[15\]](#)

Con esta información, se estima el período de amortización para el software en 2.190 días (aproximadamente 6 años) y el período de amortización para el hardware en 2.920 días (aproximadamente 8 años). A continuación, se detallan los recursos que se utilizarán durante el desarrollo de la plataforma. Es importante considerar que, siguiendo las directrices establecidas en el estudio de tecnologías, se priorizará el uso de código abierto y software libre, de forma que estos no incurrirán en coste de licencia. Las siguientes tablas presentan tanto la amortización (A) en días como el coste total del producto (CT) y, adicionalmente, el coste total para el proyecto (CP), calculado en base al coste por día amortizado:

$$CP = \frac{CT}{A}$$

Licencia	CT (€)	A (días)	CP (€/día)
Microsoft Windows 11 Pro	19,82	2190	0,009
Microsoft Office 2021 Pro	11,95	2190	0,005
Draw.io	0,00	2190	0,000
Visual Studio Code	0,00	2190	0,000
Docker Desktop	0,00	2190	0,000
Total			0,014

Tabla 3.3. Licencias de software externas.

Producto Hardware	Tipo	CT	A (días)	CP (€/día)
ASUS – TUF GAMING F15	Portátil	929,00	2920	0,32
AOC 24V5CE/BK	Monitor	109,99	2920	0,04
Logitech G502 Lightspeed	Ratón	96,99	2920	0,03
Total				0,39

Tabla 3.4. Productos hardware utilizados.

Servicios	Infraestructura	CT	A (días)	CP (€/día)
Docker Hub	Docker Inc.	0,00	2190	0,00
Contenedores Linux	Local	-	-	-
Base de datos MySQL	Local	-	-	-
Total				0,00

Tabla 3.5. Servicios de infraestructura.

3.2. Planificación Temporal

A continuación, se desarrollará una planificación temporal exhaustiva del proyecto, adoptando el día trabajado como unidad de medida principal. Dado que el equipo dispondrá de 5 horas diarias dedicadas al proyecto, la planificación reflejará una dedicación realista y sostenible a lo largo de la ejecución.

Como se ha descrito anteriormente, se llevará a cabo una metodología de desarrollo ágil basada en sprints. Este enfoque permite fragmentar los requisitos funcionales identificados en el [apartado 2.2.1](#) de forma que cada sprint entregue funcionalidades completas y operables. La duración total estimada del proyecto es de aproximadamente 120 días laborales, distribuidos de forma equilibrada entre las distintas fases del ciclo de vida del software.

RF	Resumen Funcionalidad	Prioridad
RF 1	Autenticación y acceso a la plataforma con credenciales institucionales.	1
RF 2	Visualización y edición de perfil personal del usuario.	2
RF 3	Gestión de asignaturas, grupos y profesores (nivel administrador).	3
RF 4	Creación y gestión de temarios jerarquizados.	2
RF 6, 7	Visualización de ejercicios disponibles y acceso a enunciados.	1
RF 8, 9, 10	Especificación de casos de prueba y configuración de restricciones.	1
RF 12	Envío de soluciones de código por estudiantes.	1
RF 13, 14, 15	Compilación, ejecución segura y validación automática de código.	1
RF 16, 17	Veredictos y retroalimentación inmediata.	1
RF 18, 19, 20	Historial de envíos, seguimiento de progreso del estudiante.	2
RF 21, 22, 23	Herramientas analíticas, estadísticas de grupo, exportación de datos.	3
RF 11	Disponibilidad temporal de ejercicios y temarios.	3

Tabla 3.6. Priorización de requisitos funcionales.

Para asignar eficazmente los requisitos a cada sprint, es necesario establecer una priorización que considere tanto la dependencia lógica entre funcionalidades como el valor que aporta cada una. Los requisitos se clasificarán con una prioridad de 1 (máxima) a 4 (mínima), considerando que el sistema debe entregar funcionalidades operativas y validables tras cada sprint.

El proyecto se dividirá en 4 sprints de duración variable, cada uno diseñado para entregar un incremento funcional significativo de la plataforma. A diferencia de proyectos más pequeños que pueden completarse en 2-3 sprints, la complejidad arquitectónica de este sistema (que integra ejecución segura de código, gestión de múltiples roles, análisis de datos y una interfaz adaptativa) justifica una estructura de sprints más extendida.

Con esto, podemos dar una descripción y estimación de tiempo para cada uno de los sprints de la aplicación. Comenzaremos con la estructuración de una base sólida de código e infraestructura en la que poder ir añadiéndole cada vez más funcionalidades (incluso después de este proyecto), con el fin de aumentar la mantenibilidad lo máximo posible.

Sprint 1: Establecer la arquitectura del sistema, configurar la base de datos inicial y proporcionar funcionalidad básica de autenticación y ejecución de ejercicios simples. Servirá de base para poder ir agregando funcionalidades futuras poco a poco, tanto para futuros sprints, como también para futuras funcionalidades cuando se finalice el proyecto. Sus objetivos principales serán:

- **Configuración inicial del proyecto:** Estructura de directorios, configuración de 'build', integración continua, etc.
- **Diseño e implementación de autenticación:** Sistema de inicio de sesión, gestión de tokens, control de sesiones, etc.
- **Modelo de datos inicial:** Identificar todas las entidades del sistema.
- **Interfaz de administración:** Panel de gestión de contenidos (crear, editar, eliminar, ...).
- **API REST preliminar:** *Endpoints* (llamadas) para autenticación, gestión de contenidos y consulta de ejercicios.

Tarea	Responsable	Duración (días)	Coste (€)
Análisis e inicialización del proyecto	IR	2	287,20
Diseño arquitectónico y diagramas	IS	2	287,20
Diseño de Base de Datos	ABD + IS	2	246,08
Implementación de autenticación	Programador	3	351,48
Implementación de API REST base	Programador	3	351,48
Interfaz de administración (prototipo)	Programador	2	234,20
Pruebas unitarias e integración	IS	2	287,20
Documentación Sprint 1	Programador	1	117,10
Subtotal Sprint 1			2161,94

Tabla 3.7. Desglose de Tareas Sprint 1.

Sprint 2: El segundo sprint se centra en el componente más crítico y complejo: la ejecución segura de código y la evaluación automática de ejercicios. Es el que más cuidado y rigurosidad va a necesitar, pues es el núcleo crítico de seguridad de la aplicación entera. Sus objetivos principales serán:

- **Sandboxing y aislamiento de código:** Implementación de contenedores Docker para ejecución aislada.
- **Motor de ejecución:** Compilación, ejecución de casos de prueba, veredictos.
- **Retroalimentación inicial:** Mensajes de error, o el propio veredicto inicial.
- **Gestión de restricciones:** Límites de CPU, memoria, tiempo de ejecución.
- **Validación y pruebas de seguridad:** Pruebas de infiltración, verificación de aislamiento.

Tarea	Responsable	Duración (días)	Coste (€)
Diseño del módulo de ejecución	ISE	2	311,84
Implementación con Docker	Programador	3	351,48
Desarrollo del motor de evaluación	Programador	3	351,48
Implementación de veredictos	IS	2	287,20
Gestión de restricciones	ISE	2	311,84
Pruebas de seguridad y sandboxing	ISE	2	311,84
Pruebas de carga y rendimiento	IS	1	143,60

Documentación Sprint 2	Programador	1	117,10
Subtotal Sprint 2			2286,38

Tabla 3.8. Desglose de Tareas Sprint 2.

Sprint 3: El tercer sprint completa la lógica educativa mediante sistemas de seguimiento, analítica y retroalimentación enriquecida. Estos objetivos son menos prioritarios, y se centran más en hacer el seguimiento a los estudiantes y generar analíticas según los resultados.

- **Panel de progreso personal:** Visualización de ejercicios completados, tasa de éxito, historial de intentos.
- **Sistema de perfiles de usuario:** Gestión de datos personales, preferencias, historial de actividad.
- **Herramientas docentes:** Panel de seguimiento grupal, identificar patrones.
- **Exportación de datos:** Generación de reportes en formato estándar.
- **Refinamiento de interfaces:** UX/UI mejorada, mejora de accesibilidad.

Tarea	Responsable	Duración (días)	Coste (€)
Diseño del sistema de progreso	IS	2	287,20
Panel de estudiante	Programador	3	351,48
Herramientas docentes	Programador	3	351,48
Exportación de datos	Programador	2	246,08
Refinamiento de interfaces	Programador	2	234,20
Optimizaciones de rendimiento	IS	2	287,20
Pruebas de usabilidad e integración	IS	2	287,20
Documentación Sprint 3	Programador	1	117,10
Subtotal Sprint 3			2161,94

Tabla 3.9. Desglose de Tareas Sprint 3.

Sprint 4: El sprint final se dedica a la validación integral del sistema, corrección de defectos y generación de documentación definitiva que sea lo más clara posible para el usuario.

- **Pruebas de aceptación:** Validación con usuarios finales (profesores, administradores).
- **Corrección de defectos:** *Bugfixing* y ajustes de requisitos no funcionales.
- **Manual de instalación:** Documentación detallada del despliegue en servidor.
- **Manual de usuario:** Guías específicas (estudiante, profesor, administrador).
- **Preparación para producción:** Configuración final, *backups*, planes de contingencia.

Tarea	Responsable	Duración (días)	Coste (€)
Pruebas de aceptación funcionales	IS	2	287,20
Pruebas de rendimiento	IS	2	287,20
Corrección de defectos	Programador	2	234,20
Manual de instalación	Programador	2	234,20
Manual de usuario (multi-rol)	Programador	2	234,20
Preparación para despliegue	ABD + ISE	1	163,92
Pruebas finales	IS	1	143,60
Documentación Sprint 4	Programador	1	117,10
Subtotal Sprint 4			1.701,62

Tabla 3.10. Desglose de Tareas Sprint 4.

Resumen Temporal del Proyecto

A continuación, un resumen consolidado de todas las fases del proyecto:

Fase	Duración (días)	Coste Total (€)
Fase previa (requisitos, planificación)	9	1.178,64
Sprint 1	18	2.161,94
Sprint 2	17	2.286,38
Sprint 3	18	2.161,94
Sprint 4	13	1.701,62
TOTAL	75	9.490,52

Tabla 3.11. Resumen temporal y de costes del proyecto.

Esta distribución de trabajo se ha diseñado considerando las dependencias naturales entre componentes (p. ej., no se puede evaluar código antes de tener el módulo de ejecución) y garantizando que en cada sprint se entregue una versión parcialmente funcional que proporcione valor incremental.

Diagrama de Gantt

A continuación, se presenta una visualización gráfica del cronograma del proyecto mediante un diagrama de Gantt, donde se muestran las relaciones de dependencia entre fases y sprints:

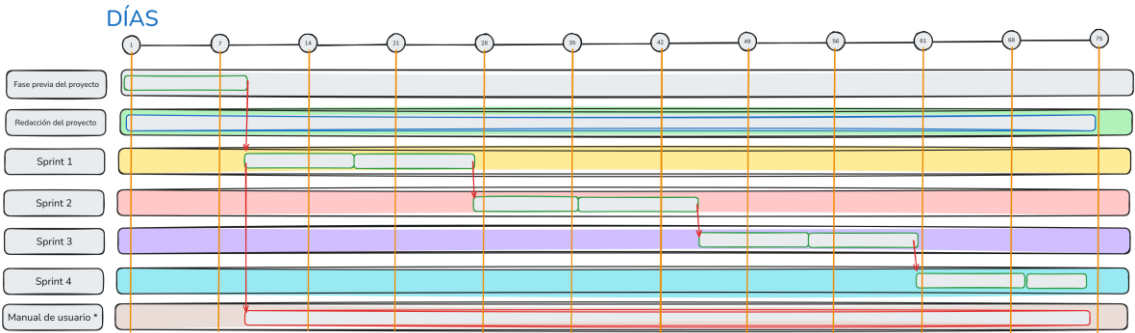


Figura 3.1. Diagrama de Gantt.

Nota: Aunque el manual de usuario forme parte del sprint 4, éste comenzará a maquetarse desde el inicio del desarrollo, para que sea más sencillo ir añadiendo funcionalidades conforme éstas vayan desarrollándose.

3.3. Estudio de las tecnologías

En primer lugar, es necesario tener en cuenta que la aplicación será alojada en un servidor institucional, con las limitaciones que esto supone. Como nuestra idea principal es usar contenedores para poder desplegar la aplicación, el uso de las tecnologías descritas a continuación no debería, en principio, suponer ningún tipo de problema; siendo el único requisito previo necesario tener instalada alguna versión de Docker. [\[20\]](#)

La selección de tecnologías constituye una decisión crítica en cualquier proyecto de software, particularmente en un sistema complejo como esta plataforma educativa que integra componentes heterogéneos: interfaz de usuario, lógica de negocio, gestión de datos y, lo más importante, ejecución segura de código arbitrario. Este apartado analiza las decisiones tecnológicas adoptadas, justificadas mediante comparación con alternativas viables y considerando las restricciones del entorno de despliegue.

Capa de Datos: Sistema Gestor de Base de Datos

Para este apartado, se ha decidido utilizar **MySQL/MariaDB**, siendo esta una alternativa ampliamente usada por la comunidad y la más común para este tipo de proyectos, donde se tiene definida de forma clara una relación entre las diferentes entidades que van a formar parte (por ejemplo, varios caos de prueba pertenecen a un ejercicio). Algunas de sus características por las que se ha elegido han sido:

- Sistema relacional de código abierto, licencia GPL.
- Amplia compatibilidad con tecnologías más adelante descritas.
- Facilita la normalización de datos y la integridad referencial.
- Rendimiento adecuado para aplicaciones de mediano tamaño.
- Estándares SQL y transacciones ACID.

Un modelo relacional garantiza estas propiedades de forma nativa, mientras que alternativas NoSQL (MongoDB, DynamoDB) requerirían lógica manual para validar integridad referencial, introduciendo riesgo de inconsistencias. Algunas de las alternativas descartadas han sido:

Alternativa	Razón de descarte
PostgreSQL	Mayor complejidad operacional; MySQL es suficiente para los requisitos.
MongoDB (NoSQL)	Modelo de datos flexible es innecesario; la estructura es predecible.
SQLite	Insuficiente para concurrencia, no soporta múltiples conexiones.
Firebase	Incrementaría costes; infraestructura actual ya proporciona MySQL.

Capa de Lógica de Negocio (Backend)

Para esta capa, que supondrá el núcleo de procesamiento de nuestra aplicación, se ha optado por utilizar un *stack* que se está haciendo nombre recientemente, que es el de **Node.js + Express + TypeScript**. Es ampliamente utilizado para APIs que requieren múltiples llamadas de

forma simultánea y que comparten gran parte de la lógica (la llamada llega, se consulta a base de datos y se manda una respuesta). Las características vienen dadas por cada uno:

Para Node.js [\[16\]](#):

- Arquitectura **event-driven** y **no bloqueante**, ideal para operaciones concurrentes.
- Amplio ecosistema de librerías (npm, con otras alternativas).

Para Express.js [\[21\]](#):

- Framework minimalista de código abierto (licencia MIT) para construir APIs REST.
- Ampliamente utilizado y documentado.
- Flexible: Permite arquitectura por capas (modelos, controladores, etc.).
- Rendimiento superior frente a otras alternativas más pesadas.

Para TypeScript:

- Es una evolución de JavaScript, agregándole tipado estático, con licencia Apache 2.0.
- Detección de errores en tiempo de compilación (antes de ejecutarse), a diferencia de JavaScript, que habría que detectarlos en tiempo de ejecución.
- Esencial para garantizar validación de datos de entrada y salida.

Aunque haya que agregar un paso extra que agrega complicación, que es el paso de compilación (de código TypeScript a JavaScript), las herramientas modernas, que vamos a aislar en contenedores, lo automatizan completamente. Algunas de las alternativas han sido:

Alternativa	Razón de descarte
Python Django	Tiempo de respuesta mayor para aplicaciones concurrentes.
Java + Spring Boot	Más adecuado para aplicaciones más complejas, con más procesado.
Golang	Comunidad menor en ámbito académico.
Fastify	Adopción menos extendida; mayor fragmentación de tutoriales.

Gestores de paquetes para Node.js

Se han evaluado tres gestores de paquetes para Node.js con el objetivo de optimizar velocidad de instalación, almacenamiento en caché y gestión de dependencias:

Gestor	npm	pnpm	bun
Velocidad de instalación	Estándar	2-3x más rápido	10x más rápido
Uso de disco	Alto	Bajo	Muy bajo
Estabilidad / madurez	Excelente	Buena	Beta
Compatibilidad	100%	~95%	~90%
Año de estreno	2010	2019	2023

Aunque para producción (docker) se usará casi con total seguridad npm (mayor compatibilidad), durante el desarrollo se optará por utilizar bun, que es una alternativa más ligera y que permite una mayor velocidad de inicialización. Se hará una prueba rápida de

velocidad con un programa simple cuando se inicie la fase de desarrollo. Además, como no se usarán dependencias viejas ni obsoletas, no será significativo el 10% de diferencia en compatibilidad que vimos en la anterior tabla.

Cabe destacar que, aunque en producción se use una alternativa y en desarrollo otra, esto no las hace incompatibles, pues servirán para compilar el proyecto e inicializarlo, sin mayores complicaciones. De hecho, durante el desarrollo se podrían usar las tres sin problema en distintas etapas, aunque por comodidad sólo se usará una. Con esto queda explicado que, tras el desarrollo de este proyecto, podrá seguirse agregando funcionalidades con el gestor de paquetes que se considere oportuno, sin lugar a incompatibilidades.

Capa de Presentación (Frontend)

Para esta capa, la más importante de cara al usuario, se ha decidido también utilizar tecnologías que estén al día. Utilizaremos una pila de tecnologías que combinan rendimiento, sin cargar demasiados *scripts* en el navegador, fluidez y dinamismo, cargando sólo los *scripts* completamente necesarios, y estilos facilitados y diseñados para el desarrollo: **Astro + React + TailwindCSS**. Las características de cada uno son:

Para Astro [\[18\]](#):

- Framework de construcción de sitios estáticos/híbridos, licencia MIT.
- Cero JavaScript por defecto (mejora rendimiento de páginas estáticas como home, perfil, temarios).
- Integración perfecta con React para componentes dinámicos.
- Soporta SSR (Server-Side Rendering) para contenido dinámico.
- Especializado en generación de HTML estático con renderizado selectivo de componentes interactivos.

Para React [\[17\]](#):

- Biblioteca de JavaScript para construir interfaces interactivas, licencia MIT.
- Modelo de componentes basado en Virtual DOM para actualizaciones eficientes.
- Gran comunidad y ecosistema.

Para TailwindCSS [\[19\]](#):

- Framework de CSS utilitario, licencia MIT.
- Alternativa moderna a Bootstrap, enfocada en composición mediante clases pequeñas y reutilizables.
- Pueden diseñarse etiquetas personalizadas según las necesidades de desarrollo.

Este tipo de pilas, para el caso de Astro y React, necesitan lógica para cada componente, que en este caso también utilizaremos TypeScript, agregando de nuevo la capa de complejidad descrita anteriormente, pero que no supondrá ningún problema a la hora de la compilación debido al uso de Docker. Algunas alternativas que se han tenido en cuenta:

Alternativa	Razón de descarte
Vue.js + Nuxt.js	Comunidad menor en industria. Astro tiene mejor soporte para SSG.
Angular	Muy complejo para este proyecto; requiere aprendizaje pronunciado.
SvelteKit	Comunidad aún pequeña; menos documentación y librerías de terceros.
Next.js	Opción viable, pero Astro es más ligero y especializado para este caso.

Capa de Ejecución Segura de Código

Este componente es el más crítico del proyecto, dado que ejecuta código arbitrario enviado por usuarios (potencialmente malicioso). La arquitectura debe garantizar:

- Aislamiento completo respecto al sistema anfitrión.
- Límites de recursos (CPU, memoria, tiempo).
- Prevención de acceso a red, sistema de archivos o procesos del host.

Utilizaremos una pila de desarrollo simple, basada en Docker y Linux Containers siendo que cada envío se ejecute en un contenedor efímero (de vida corta) basado en una imagen predefinida, que variará dependiendo del lenguaje. Con esto garantizamos seguridad, por ejemplo, aislando el PID de los procesos, el hostname, la red, etc.

Capa Adicional: Autenticación

Para la autenticación, se ha decidido separarla a una capa adicional, independiente del resto del backend, para poder utilizar una biblioteca de código abierto y altamente probada, con el motivo de no tener que implementar nosotros de cero dicho sistema. La opción seleccionada ha sido **Authgear Server** [22], que dispone de numerosas ventajas:

- Plataforma de autenticación y gestión de identidades de código abierto, licencia Apache.
- Código abierto y público en su [repositorio](#).
- Soporte para múltiples métodos de autenticación: credenciales locales, OAuth 2.0, OpenID Connect, SAML, SSO institucional.
- Control total: Al ser autoalojado, los datos de autenticación permanecen en servidores del Departamento (cumple RGPD sin problemas).

Conclusiones y Esquema General

Dado que todas las tecnologías que hemos descrito en las diferentes capas del proyecto son de código abierto y no precisan de ningún tipo de suscripción adicional, no será necesario volver a hacer una estimación de los costes..

El esquema que se muestra a continuación muestra como queda, provisionalmente, la infraestructura del proyecto. Esto queda sujeto a futuros cambios, según se vaya viendo conveniente durante el desarrollo de la aplicación, por ejemplo, separar por completo la capa de autenticación de la API REST.

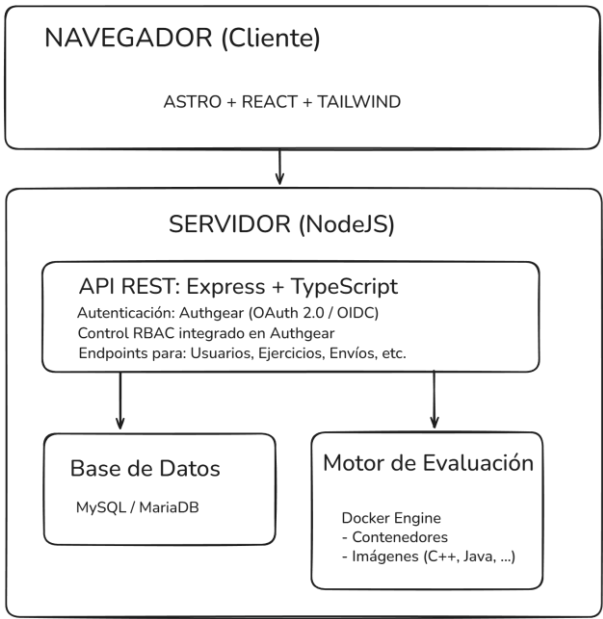


Figura 3.3.1. Prototipo de infraestructura del proyecto.

3.4. Estimación de costes

Categoría	Coste total (€)	% Presupuesto
Recursos Humanos	9.490,52	99,7%
Recursos Hardware	28,74	0,3%
Recursos Software	1,04	0,0%
Infraestructura	0,00	0,0%
Tecnologías OSS	0,00	0,0%
TOTAL	9.520,3	100%

Tabla 3.4.1. Coste total del proyecto.

Bibliografía

- [0] Wasik, S., Antczak, M., Badura, J., Laskowski, A., & Sternal, T. (2018). A survey on online judge systems and their applications. *ACM Computing Surveys*, 51(1), Article 3, 1–34. Recuperado de <https://doi.org/10.1145/3143560>.
- [1] Paiva, J. C., Leal, J. P., & Figueira, Á. (2022). Automated Assessment in Computer Science Education: A State-of-the-Art Review. *ACM Transactions on Computing Education*, 22(3). Recuperado de <https://repositorio.inesctec.pt/server/api/core/bitstreams/49605368-51db-4de7-97fa-de7922b28790/content>.
- [2] Horváth, M., Kormaník, T., & Porubán, J. (2024). Adaptation of Automated Assessment System for Large Programming Courses. En 5th International Computer Programming Education Conference (ICPEC 2024) (pp. 4:1–4:11). Schloss Dagstuhl – Leibniz-Zentrum für Informatik (OASlcs, Vol. 122). Recuperado de <https://drops.dagstuhl.de/storage/01oasics/oasics-vol122-icpec2024/OASlcs.ICPEC.2024.4/OASlcs.ICPEC.2024.4.pdf>.
- [3] Hugging Face. (n.d.). Secure code execution. smolagents documentation. Recuperado de https://huggingface.co/docs/smolagents/en/tutorials/secure_code_execution.
- [4] Vargas-Enríquez, J., García-Mundo, L., Genero, M., & Piattini, M. (2015). Análisis de uso de la Gamificación en la Enseñanza de la Informática. En *Actas de las XXI Jornadas de la Enseñanza Universitaria de la Informática (JENUI)*, Andorra la Vella, 8–10 de julio de 2015. Recuperado de <https://upcommons.upc.edu/server/api/core/bitstreams/36925bc1-1407-45a8-822f-1d2a52c9e308/content>.
- [5] Moodle. (n.d.). Moodle.org. Recuperado de <https://moodle.org/?lang=es>.
- [6] Amazon Web Services. (n.d.). ¿Cuál es la diferencia entre el front end y back end en el desarrollo de aplicaciones? Recuperado de <https://aws.amazon.com/es/compare/the-difference-between-frontend-and-backend/>.
- [7] Wikipedia. (n.d.). Especificación de requisitos de software. Recuperado de https://es.wikipedia.org/wiki/Especificaci%C3%B3n_de_requisitos_de_software.
- [8] IEEE. (1998). IEEE Std 830-1998: Recommended Practice for Software Requirements Specifications (versión en español, Universidad de Cantabria). Recuperado de https://www.ctr.unican.es/asignaturas/is1/IEEE830_esp.pdf.
- [9] Amazon Web Services. (n.d.). ¿Cuál es la diferencia entre SSL y TLS? Recuperado de <https://aws.amazon.com/es/compare/the-difference-between-ssl-and-tls/>.
- [10] BeChallenge. (n.d.). Cómo evaluar plataformas educativas en la nube. Recuperado de <https://blog.bechallenge.io/como-evaluar-plataformas-educativas-en-la-nube/>.

- [11] Parlamento Europeo y Consejo de la Unión Europea. (2016). Reglamento (UE) 2016/679 del Parlamento Europeo y del Consejo de 27 de abril de 2016 relativo a la protección de las personas físicas en lo que respecta al tratamiento de datos personales y a la libre circulación de estos datos y por el que se deroga la Directiva 95/46/CE (Reglamento General de Protección de Datos). Diario Oficial de la Unión Europea, L 119, 1–88. Recuperado de <https://www.boe.es/doue/2016/119/L00001-00088.pdf>.
- [12] IONOS. (n.d.). ISO 8601: expresiones temporales inequívocas. Digitalguide. Recuperado de <https://www.ionos.es/digitalguide/paginas-web/desarrollo-web/iso-8601/>.
- [13] W3C Web Accessibility Initiative (WAI). (n.d.). Sumario de WCAG 2. Recuperado de <https://www.w3.org/WAI/standards-guidelines/wcag/es>.
- [14] es.talent.com. (2024). Salario medio por trabajo en España, 2024. Recuperado de <https://es.talent.com/salary?job=2024>.
- [15] BOE. (2014). Ley 27/2014, de 27 de noviembre, del Impuesto sobre Sociedades. Recuperado de <https://www.boe.es/buscar/act.php?id=BOE-A-2014-12328>.
- [16] Node.js. (n.d.). Documentation. Recuperado de <https://nodejs.org/en/docs/>.
- [17] React. (n.d.). Página de inicio. Recuperado de <https://react.dev>.
- [18] Astro Documentation. (n.d.). @astrojs | Docs. Recuperado de <https://docs.astro.build/en/getting-started/>.
- [19] Astro Documentation. (n.d.). Integrating Tailwind CSS | Docs. Recuperado de <https://docs.astro.build/en/guides/integrations-guide/tailwind/>.
- [20] Docker Documentation. (n.d.). Orientation and setup. Recuperado de <https://docker-docs.uclv.cu/get-started/>.
- [21] Express.js. (n.d.). Express: Node.js web application framework. Recuperado de <https://expressjs.com/>.
- [22] Authgear. (n.d.). authgear/authgear-server: Open source alternative to Auth0 / Firebase Auth [Repositorio GitHub]. Recuperado de <https://github.com/authgear/authgear-server>.